



Full length article

Advancing software security: DCodeBERT for automatic vulnerability detection and repair

Ahmed Bensaoud^{*,}, Jugal Kalita

Department of Computer Science, University of Colorado Colorado Springs (UCCS), Colorado Springs, CO, USA

ARTICLE INFO

Keywords:

Large Language Models
Adversarial attacks
Vulnerability detection
Data privacy
Natural language processing

ABSTRACT

The exponential growth of software complexity has led to a corresponding increase in software vulnerabilities, necessitating robust methods for automatic vulnerability detection and repair. This paper proposes DCodeBERT, a large language model (LLM) fine-tuned for vulnerability detection and repair in software code. Leveraging the pre-trained CodeBERT model, DCodeBERT is designed to understand both natural language and programming language context, enabling it to effectively identify vulnerabilities and suggest repairs. We conduct experiments to evaluate DCodeBERT's performance, comparing it against several baseline models. The results demonstrate that DCodeBERT outperforms the baselines in both vulnerability detection and repair tasks across multiple programming languages, showcasing its effectiveness in enhancing software security.

1. Introduction

Large Language Models (LLMs) have become a cornerstone of modern Natural Language Processing (NLP), enabling significant advancements in language-related tasks such as text generation, translation, summarization, and sentiment analysis. Examples of these models include OpenAI's GPT series, Google's BERT, and Meta's LLaMA [1–4]. Trained on massive datasets, LLMs can generate human-like text, understand context, and even exhibit rudimentary reasoning capabilities. Due to their transformative potential, the widespread adoption of LLMs has introduced new security and privacy challenges that need careful consideration.

The primary security concern associated with LLMs is their vulnerability to adversarial attacks. These attacks can manipulate the inputs to the model in subtle ways that cause it to produce incorrect or harmful outputs. For instance, adversarial examples can be crafted to exploit specific weaknesses in the model's architecture, leading to erroneous or malicious outputs [5–8]. Such vulnerabilities have been demonstrated across various types of models and tasks, highlighting a significant risk for applications relying on LLMs [9–11].

Furthermore, LLMs trained on biased or sensitive data can inadvertently perpetuate harmful stereotypes or expose private information. This issue has been extensively discussed in the literature, with studies showing that models can amplify existing biases present in their training data [12–14]. Additionally, the data used to train these models often include personal information, which raises significant privacy concerns. Unauthorized access to or misuse of such models can lead to breaches of privacy and confidentiality [15–17].

The implications of these vulnerabilities are profound, especially given the increasing integration of LLMs into critical applications across various industries. Researchers have highlighted the potential risks associated with the misuse of LLMs, including the generation of harmful content, misinformation, and the facilitation of cyber-attacks [18,19]. These risks necessitate the development of robust mitigation strategies to enhance the security and privacy of LLMs.

To address these challenges, various approaches have been proposed. Techniques such as adversarial training, robust optimization, differential privacy, and data sanitization have shown promise in enhancing the resilience of LLMs against adversarial attacks and protecting sensitive information [20]. However, these methods often involve trade-offs between security, performance, and computational overhead, underscoring the need for ongoing research and innovation.

Recent studies have also explored the capabilities of LLMs in specific contexts, such as vulnerability detection and repair in software systems. For example, Pearce et al. [21] examined zero-shot vulnerability repair using LLMs, while Steenhoek et al. [22] conducted a comprehensive study of LLMs' capabilities for vulnerability detection. These studies demonstrate the potential of LLMs to enhance cybersecurity practices, though they also reveal limitations that need to be addressed.

Yao et al. [23] conducted a survey on LLM security and privacy, categorizing the issues into the good, the bad, and the ugly, and providing a holistic overview of the current state of research. Zhou et al. [24] presented emerging results and future directions for using LLMs in vulnerability detection, highlighting the ongoing evolution of this field.

* Corresponding author.

E-mail address: abensaou@uccs.edu (A. Bensaoud).

The major contributions of our study are:

- Development of DCodeBERT, a transformer-based model specifically designed for vulnerability detection and repair.
- Implementation of multi-task learning to simultaneously train the model for both vulnerability detection and repair tasks.
- Utilization of pre-training objectives such as Masked Language Modeling (MLM) and Replaced Token Detection (RTD) to enhance the model's understanding of code.
- Evaluation of the model using various metrics for both vulnerability detection and repair tasks, demonstrating its effectiveness in identifying and repairing vulnerabilities in code.

The remainder of this paper is organized as follows: Section 2 provides a detailed review of related work. Section 3 describes the methodology, including our model architecture and experimental setup. Section 4 presents the experimental results and analysis. Section 5 discusses the implications of our findings and potential future work. Finally, Section 6 concludes the paper.

2. Related work

2.1. Large language models in vulnerability detection

The application of large language models (LLMs) in software vulnerability detection has garnered significant attention in recent years. Pearce et al. [21] explored the zero-shot capabilities of LLMs, specifically examining their ability to repair vulnerabilities without prior domain-specific training. Their findings indicate that while LLMs can identify and suggest fixes for certain types of vulnerabilities; their performance is inconsistent across different contexts and vulnerability types.

Steenhoek et al. [22] conducted a comprehensive study on the capabilities of LLMs for vulnerability detection. Their research highlighted the strengths and limitations of various models, emphasizing the need for fine-tuning and domain-specific adjustments to enhance detection accuracy. They concluded that while LLMs hold promise, there is a significant gap in their ability to detect complex and nuanced vulnerabilities compared to traditional static and dynamic analysis tools.

Zhou et al. [24] investigated the effectiveness of large pre-trained language models (LLMs) like GPT-3.5 and GPT-4 in software vulnerability detection, finding that GPT-4 excels in this task. It highlighted promising future research directions and provides a publicly available replication package for further study.

Yang et al. [25] proposed LLMAO, a fault localization approach leveraging large language models to identify buggy lines of code without test coverage information. Shi et al. [26] presented Avatar, an approach that optimizes large language models of code to be compact, efficient, and energy-saving for deployment on developers' devices, addressing issues like restricted internet access and privacy policies.

McIntosh et al. [27] empirically investigated the semantic vulnerabilities of four popular pre-trained commercial Large Language Models to ideological manipulation, revealing that LLMs are susceptible to targeted semantic exploits due to limitations in comprehending detailed linguistic variations. They highlighted the limitations of Reinforcement Learning from Human Feedback (RLHF) in mitigating ideological conditioning prompts and adequately representing diverse human values, often reflecting the values of the controlling entities. The findings underscore the universal ideological vulnerability in LLMs and call for a multidisciplinary approach to develop more robust and ethical AI systems.

2.2. Security and privacy concerns

The security and privacy implications of deploying LLMs in vulnerability detection are critical considerations. Yao et al. [23] provide an extensive survey on LLM security and privacy, categorizing the threats and vulnerabilities associated with these models. Their work underscores the importance of developing robust mitigation strategies to protect against adversarial attacks and data leakage, which are prevalent concerns in the use of LLMs for security applications.

In a similar vein, Abdali et al. [28] examined the various threats and vulnerabilities inherent to LLMs. They proposed a set of responsible practices for deploying LLMs in security-critical environments, emphasizing the need for continuous monitoring and updating of models to safeguard against evolving threats.

Chen et al. [29] addressed the security, privacy, and copyright risks associated with large language models (LLMs), alongside recent solutions to these issues. They explore adversarial threats during LLM training, defenses against malicious inference-time attacks, privacy protection for users and LLM decisions, and copyright safeguarding, while also assessing how to prevent information leakage and control improper content usage. By answering these critical questions, the tutorial highlighted emerging challenges and provided participants with tools and knowledge to ensure the responsible and secure use of LLMs.

Das et al. [30] highlighted the significant capabilities of Large Language Models in language processing tasks but emphasized their vulnerability to security and privacy attacks, such as data poisoning, jailbreaking, and PII leakage. They provided a comprehensive review of these challenges and the existing defense mechanisms while assessing application-specific risks in domains like transportation, education, and healthcare. Their research concluded by identifying research gaps and advocating for further exploration into secure and privacy-preserving techniques to ensure the safe and reliable deployment of LLMs.

Wu et al. [31] systematically examined the security vulnerabilities of Large Language Model systems, focusing on OpenAI GPT-4, by analyzing the flow of information within the model and its interactions with external components. Despite OpenAI's implementation of safety constraints, the study identified multiple exploitable weaknesses, including the circumvention of safeguards such as Safe URL Checks, demonstrating real-world threats such as illicit extraction of user chat histories through carefully designed attacks.

2.3. Emerging techniques and tools

Several researchers have explored novel techniques and tools to enhance the effectiveness of LLMs in vulnerability detection. Zhou et al. [24] discussed emerging results and future directions in the application of LLMs for vulnerability detection. They highlighted the potential of integrating LLMs with traditional static analysis tools to improve detection rates and reduce false positives.

Mathews et al. [32] introduced LLbezpeky, a framework leveraging LLMs for vulnerability detection. Their approach combined LLMs with heuristic-based methods to identify and prioritize vulnerabilities in large codebases, demonstrating improved detection performance over standalone LLMs.

Zhang et al. [33] investigated the evolving field of knowledge editing for Large Language Models, emphasizing the need for efficient and adaptive methods to modify model behavior post-training. They categorized editing techniques into three groups – resorting to external knowledge, merging knowledge into the model, and editing intrinsic knowledge – providing a unified framework for analysis. Through extensive empirical evaluations and the introduction of the KnowEdit benchmark, the authors highlighted both the potential and challenges of knowledge editing, underscoring its impact on model performance, usability across tasks, and the underlying mechanisms governing LLMs' cognitive processes.

Fields et al. [34] provided a comprehensive survey of text classification techniques with Large Language Models, with a specific focus on transformer-based models. The study categorizes these techniques across applications such as sentiment analysis and chatbot-driven question-answering, highlighting the trend towards multimodal classification. Through rigorous empirical evaluations and the introduction of the KnowEdit benchmark, the research illuminated the diverse performance of LLMs across 358 datasets, thereby challenging assumptions regarding their universal superiority. Furthermore, the study addresses ethical implications and advocates for ongoing research in model interpretability and fairness.

2.4. Vulnerability repair and mitigation

The repair and mitigation of vulnerabilities using LLMs have also been a focal point of recent studies. Zhou et al. [24] presented a literature review and roadmap for utilizing LLMs in vulnerability detection and repair. Their work outlined the challenges and potential solutions in automating the repair process, emphasizing the role of contextual understanding in generating accurate and effective fixes.

Noever [35] explored the dual capabilities of LLMs in both finding and fixing vulnerable software. His research highlighted the initial successes and persistent challenges in automating the vulnerability repair process, noting the variability in model performance across different programming languages and vulnerability types.

Ahmad et al. [36] investigated the use of Large Language Models like OpenAI's Codex for automatically repairing security-relevant bugs in hardware designs, focusing on Verilog code. They demonstrated that an ensemble of LLMs can outperform state-of-the-art automated tools in fixing these bugs, providing a framework for evaluating and optimizing LLM parameters for effective bug repair.

Le et al. [37] evaluated the effectiveness of Large Language Models (LLMs), specifically ChatGPT and Bard, in automatically finding and fixing security vulnerabilities in JavaScript code. The results demonstrate that both models perform well, with accuracy rates of 71.66% for ChatGPT and 68.33% for Bard, and that providing more context in the prompts significantly improves their performance, enhancing accuracy by up to 55%.

Omari et al. [38] evaluated ChatGPT's ability to detect and repair bugs in Python code using benchmarks such as QuixBugs and the Goodrich textbook. They showed that ChatGPT can accurately fix 50% of documented code, with performance dropping to 25% for obfuscated code and 15% when both documentation is removed and code is obfuscated, outperforming traditional Automated Program Repair tools.

2.5. Advanced evaluation metrics and methodologies

To accurately assess the performance of LLMs in vulnerability detection and repair, researchers have proposed various evaluation metrics and methodologies. Ding et al. [39] conducted an empirical study on the state of vulnerability detection with code language models. They utilized a comprehensive set of metrics, including precision, recall, and F1-score, to evaluate model performance and identify areas for improvement.

Yang et al. [40] proposed DLAP, a deep learning augmented LLM prompting framework designed to enhance vulnerability detection. Their framework leverages advanced evaluation techniques to refine model prompts and improve detection accuracy, demonstrating the potential for adaptive and context-aware LLMs in security applications.

Tosi [41] evaluated AI-generated code from three Large Language Models on complex Java programming problems. They revealed that while LLMs like GPT-3, GPT-4, and Bard can produce functional and quality code for problems with available solutions, human supervision is essential to ensure correctness and adherence to coding standards. They identified the need for clear prompts and expert oversight, suggesting a collaborative approach between AI and software professionals to enhance code quality and reliability.

Osman et al. [42] investigated the application of automatic ontology matching techniques to integrate non-semantically enriched relational datasets, focusing on the spatio-temporal alignment of macro indicators. Despite employing existing tools, the authors revealed significant uncertainty and inaccuracies in the automated matching process due to the lack of semantic enrichment in the original datasets. The results highlighted the necessity for improved methods in knowledge engineering to reduce uncertainty and enhance the dependability of data integration systems, particularly with advancements in technology towards more advanced AI solutions such as large language models.

Yang et al. [43] introduced InterCode, a framework designed for interactive code generation, which integrates language models with environments that mimic human coding practices. By providing secure, reproducible execution environments through Docker, InterCode facilitates interactive coding tasks in Bash, SQL, and Python. The framework serves as a versatile benchmark for evaluating language models, showcasing the benefits of interactive feedback in enhancing code generation capabilities and enabling the creation of new, multi-step coding challenges like Capture the Flag. Future directions include expanding to support additional programming languages and datasets to further extend InterCode's applicability and effectiveness in interactive coding scenarios.

Shen et al. [44] provided a foundational survey that examines various machine learning techniques applied to the analysis of encrypted network traffic. This survey highlighted the significant challenges that encryption introduces to traditional traffic analysis methods. It reviewed the workflow of encrypted traffic analysis, including stages such as traffic collection, representation, and performance evaluation, while classifying existing studies based on their goals, such as network asset identification, privacy leakage detection, and anomaly detection. The survey emphasized the need for innovative approaches to address the balance between security and privacy concerns in encrypted traffic analysis.

Building upon these insights, Ibraheem et al. [45] presented a machine learning-based approach specifically for detecting anomalies in encrypted HTTPS traffic. They compared various feature selection techniques to enhance the performance of their anomaly detection models. They demonstrated that their proposed approach, using the Random Forest model, achieved high accuracy in identifying anomalous traffic, surpassing existing methods. However, the study also acknowledged the limitations of their approach, suggesting the need for further optimization and broader evaluation.

Hwang et al. [46] introduced D-PACK, a deep learning framework aimed at the early detection of malicious traffic within IoT networks. Their framework combined a Convolutional Neural Network with an autoencoder to automatically profile traffic patterns. D-PACK achieved nearly 99% accuracy with a low false-positive rate by inspecting only the first two packets and 80 bytes per packet in each flow, thus significantly reducing processing time and improving detection speed.

Fotiadou et al. [47] designed a deep learning-based approach for network intrusion detection that utilizes Convolutional Neural Networks and Long Short-Term Memory networks to analyze network logs from the pfSense firewall. Their proposed models effectively classified different event types in the logs, demonstrating high accuracy in detecting network anomalies. They highlighted the potential of these deep learning architectures for broader applications in network security, with minimal preprocessing required for adaptation to various intrusion detection scenarios.

2.6. Comparative analyses and surveys

Comparative analyses and surveys provide valuable insights into the effectiveness of LLMs relative to other approaches. Chowdhury et al. [48] conducted a comparative survey of attacks on LLMs, providing a detailed taxonomy of attack vectors and defense mechanisms. Their

work is instrumental in understanding the security landscape and the resilience of LLMs to various threats.

Hassanin and Moustafa [49] offered a comprehensive overview of LLMs for cyber defenses, identifying key opportunities and future research directions. Their survey highlights the evolving role of LLMs in enhancing cybersecurity measures and the critical challenges that need to be addressed to fully realize their potential.

Goto et al. [50] conducted a comprehensive evaluation of four prominent Large Language Models – Google Gemini, Mistral 8x7B, ChatGPT-4, and Microsoft Phi-1.5 – using the Microsoft PromptBench dataset to assess their resilience against various adversarial conditions, including syntactic manipulations, semantic alterations, and contextually misleading cues. The authors demonstrated significant variability in the models' abilities to withstand adversarial challenges, revealing distinct strengths and weaknesses across different forms of linguistic manipulation.

Ullah et al. [51] introduced SecLLMHolmes, a fully automated framework designed to assess Large Language Models for their capability to identify security-related bugs in software code. Through the evaluation of eight state-of-the-art LLMs across 228 code scenarios and using 17 prompting techniques, they revealed significant shortcomings in the performance of current LLMs, including nondeterministic responses, incorrect reasoning, and poor adaptability to real-world software vulnerabilities.

3. Methodology

Our methodology for leveraging large language models (LLMs) for vulnerability detection and repair involves several key steps: data preparation, model architecture design, training, and evaluation. In this section, we outline each step in detail and present the mathematical foundations that underlie our approach.

3.1. Data preparation

The dataset used for training consists of code snippets which are labeled with the specific type. For evaluation a testing sample is classified as either vulnerable or non-vulnerable. Additionally, the dataset includes pairs of vulnerable and repaired code snippets for the vulnerability repair task.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be the dataset, where x_i represents a code snippet and $y_i \in \{0, 1\}$ is the corresponding label indicating the presence of a vulnerability. For the repair task, we have pairs $\mathcal{R} = \{(x_i, z_i)\}_{i=1}^M$, where x_i is a vulnerable code snippet and z_i is its repaired version.

3.2. Model architecture

Our model, DCodeBERT, is based on a transformer architecture specifically designed for code understanding tasks. We use a pre-trained language model CodeBERT [52], and fine-tune it for both vulnerability detection and repair. We introduce ten joint embeddings to better capture the semantics of both natural language (NL) and programming language (PL).

3.2.1. Transformer architecture

The transformer model consists of an encoder-decoder architecture [53]. The encoder processes the input sequence, while the decoder generates the output sequence. The encoder is composed of multiple layers of self-attention and feed-forward neural networks.

Mathematically, the self-attention mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q (queries), K (keys), and V (values) are linear projections of the input sequence, and d_k is the dimension of the keys.

Each encoder layer consists of a multi-head self-attention mechanism followed by a position-wise feed-forward network. The output of the attention mechanism is given by:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, W_i^Q , W_i^K , W_i^V , and W^O are learned projection matrices.

The position-wise feed-forward network is defined as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

where W_1 and W_2 are weight matrices, and b_1 and b_2 are biases (see Fig. 1).

3.2.2. Pre-training objectives

We employ two pre-training objectives to enhance the model's understanding of code: Masked Language Modeling (MLM) and Replaced Token Detection (RTD).

- Masked Language Modeling (MLM): In MLM, a random set of positions are selected for both NL and PL to mask out, and then replace the selected positions with a special [MASK] token. The MLM objective is to predict the original tokens which are masked out.

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \text{masked}} \log P(x_i | x_{\setminus i})$$

where x_i is the masked token, and $x_{\setminus i}$ is the input sequence with the token at position i masked out.

- Replaced Token Detection (RTD): In RTD, a generator model first randomly masks out a few tokens in the original NL and PL sequences. The generator then predicts possible replacements for these masked tokens. Subsequently, a discriminator model is trained to determine whether each token in the sequence is the original one or has been replaced by the generator, making it a binary classification problem.

$$\mathcal{L}_{\text{RTD}} = - \sum_{i=1}^T [y_i \log P(y_i | x_i) + (1 - y_i) \log(1 - P(y_i | x_i))]$$

where y_i is a binary indicator that the token x_i is replaced or not.

3.3. Training

We employ a multi-task learning approach, where the model is trained simultaneously on both detection and repair tasks. This allows the model to leverage shared representations and improve generalization across tasks.

3.3.1. Initial multi-task learning framework

In multi-task learning, a single model is trained on multiple tasks simultaneously, with the goal of improving performance by leveraging commonalities and differences across tasks. We define our multi-task learning problem with the following loss functions and optimization equations.

The overall training objective combines the loss functions for the detection and repair tasks. The total loss is a weighted sum of the individual task losses:

$$\mathcal{L} = \lambda_{\text{detection}} \mathcal{L}_{\text{detection}} + \lambda_{\text{repair}} \mathcal{L}_{\text{repair}}$$

where $\lambda_{\text{detection}}$ and λ_{repair} are hyperparameters that control the relative importance of each task.

Detection task. For the vulnerability detection task, we use the binary cross-entropy loss:

$$\mathcal{L}_{\text{detection}} = - \frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i$ is the predicted probability that the code snippet x_i is vulnerable, and y_i is the true label.

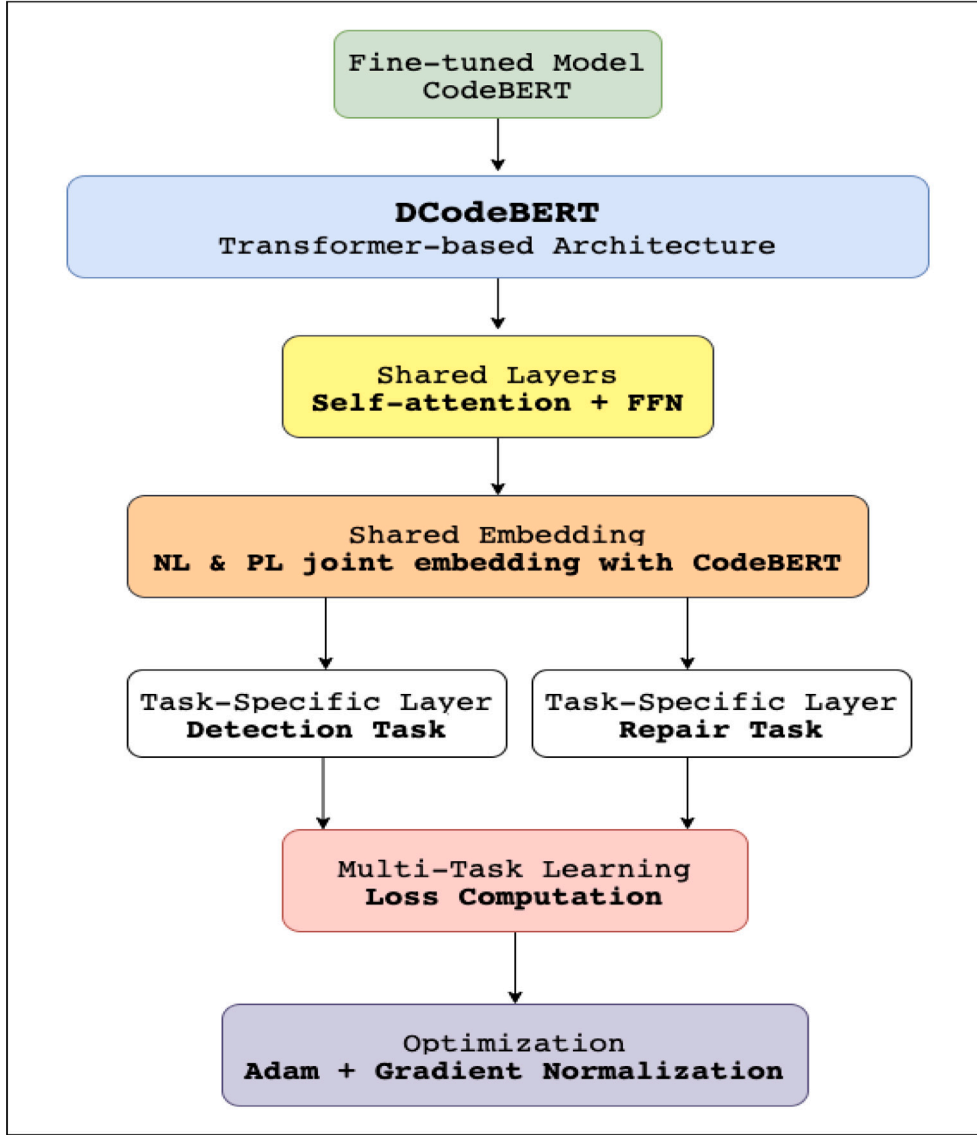


Fig. 1. DCodeBERT model architecture.

Repair task. For the vulnerability repair task, we use the mean squared error (MSE) loss between the predicted repaired code snippet and the target repaired snippet:

$$\mathcal{L}_{\text{repair}} = \frac{1}{M} \sum_{i=1}^M \|\hat{z}_i - z_i\|^2$$

where $\hat{z}_i$ is the predicted repaired code snippet, and z_i is the true repaired code snippet.

Optimization. We optimize our model parameters using the Adam optimizer, which adapts the learning rate based on estimates of lower-order moments. The update rule for the parameters is:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where η is the learning rate, $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second moments of the gradients, respectively.

3.3.2. Updated multi-task learning models

To further enhance our multi-task learning framework, we modified the task-specific layers, shared-private, and gradient normalization.

Task-specific layers. We introduce task-specific layers on top of the shared base model to capture task-specific features. Let f_{shared} represent the shared layers and f_{task_i} represent the task-specific layers for task i :

$$h_{\text{shared}} = f_{\text{shared}}(x)$$

$$\hat{y}_i = f_{\text{task}_i}(h_{\text{shared}})$$

where h_{shared} is the shared representation and $\hat{y}_i$ is the prediction for task i .

Shared-private. The shared-private model architecture consists of a shared network for learning common features and private networks for learning task-specific features. Let f_{shared} and f_{private_i} denote the shared and private networks, respectively:

$$h_{\text{shared}} = f_{\text{shared}}(x)$$

$$h_{\text{private}_i} = f_{\text{private}_i}(x)$$

$$\hat{y}_i = f_{\text{task}_i}([h_{\text{shared}}, h_{\text{private}_i}])$$

where $[h_{\text{shared}}, h_{\text{private}_i}]$ denotes the concatenation of shared and private representations.

Gradient normalization. To ensure balanced learning across tasks, we apply gradient normalization techniques. One such technique is the Dynamic Weight Averaging (DWA) method, which adjusts the task weights based on the rate of change in the task losses:

$$\lambda_{\text{task}_i} = \frac{N \cdot \exp\left(\frac{L_{\text{task}_i}(t-1)}{L_{\text{task}_i}(t-2)}\right)}{\sum_{j=1}^N \exp\left(\frac{L_{\text{task}_j}(t-1)}{L_{\text{task}_j}(t-2)}\right)}$$

where $L_{\text{task}_i}(t)$ denotes the loss for task i at epoch t .

Uncertainty weighting. We also incorporate uncertainty-based weighting to dynamically adjust task weights. Let σ_i denote the uncertainty for task i , estimated during training. The uncertainty-based task weight is given by:

$$\lambda_{\text{task}_i} = \frac{1}{2\sigma_i^2}$$

The total loss then incorporates these uncertainty weights:

$$\mathcal{L} = \sum_{i=1}^N \left(\frac{1}{2\sigma_i^2} \mathcal{L}_{\text{task}_i} + \log \sigma_i \right)$$

We enhance the robustness and performance of our model in handling both vulnerability detection and repair tasks, leading to more accurate and reliable predictions.

We trained DCodeBERT by iteratively updating its parameters across epochs, focusing on both vulnerability detection and repair tasks. It utilizes multi-task learning with task-specific losses, dynamically adjusts task weights using Dynamic Weight Averaging (DWA), and optimizes model parameters using the Adam optimizer to enhance overall performance (see Algorithm 1).

Algorithm 1 DCodeBERT Training Algorithm with Multi-Task Learning and Gradient Normalization

```

1: procedure TRAIN( $\mathcal{D}, \mathcal{R}$ , epochs,  $\lambda_{\text{detection}}, \lambda_{\text{repair}}$ )
2:   Initialize DCodeBERT model parameters
3:   for epoch  $\leftarrow 1$  to epochs do
4:     for  $(x_i, y_i) \in \mathcal{D}$  do
5:       Compute  $\mathcal{L}_{\text{detection}}$  using  $x_i$  and  $y_i$ 
6:       Update model parameters using Adam optimizer
7:     end for
8:     for  $(x_i, z_i) \in \mathcal{R}$  do
9:       Compute  $\mathcal{L}_{\text{repair}}$  using  $x_i$  and  $z_i$ 
10:      Update model parameters using Adam optimizer
11:    end for
12:    Compute total loss  $\mathcal{L} = \lambda_{\text{detection}} \mathcal{L}_{\text{detection}} + \lambda_{\text{repair}} \mathcal{L}_{\text{repair}}$ 
13:    Apply Dynamic Weight Averaging (DWA) to update task
    weights
14:    Update model parameters using  $\mathcal{L}$  and Adam optimizer
15:  end for
16:  return Trained DCodeBERT model
17: end procedure

```

The fine-tuning process for DCodeBERT helps maximized its effectiveness in detecting and repairing vulnerabilities in code. We utilized a diverse training dataset $\mathcal{D}_{\text{train}}$, which includes labeled code snippets for vulnerability detection and paired examples of vulnerable and repaired code snippets for the repair task. This dataset is carefully curated to cover a broad range of programming languages and vulnerability types, ensuring the model is exposed to a variety of real-world scenarios.

DCodeBERT was initialized with pre-trained weights from CodeBERT and then fine-tuned in a multi-task learning framework that simultaneously optimized for both detection and repair tasks. During this process, the model was trained over 10 epochs with a batch size of 32. The learning rate η was set to 1×10^{-5} and managed by a linear scheduler with a warmup phase, followed by a decay phase to stabilize

training. The optimizer used was AdamW, which was configured with a weight decay of 0.01 to prevent overfitting. Additionally, gradient clipping was applied with a maximum norm of 1.0 to ensure that the gradients remain stable throughout the training process. Dynamic weighting techniques and gradient normalization were employed to balance the learning between the tasks. The loss function $\mathcal{L}$ combined detection and repair losses, weighted by $\lambda_{\text{detection}}$ and λ_{repair} coefficients, respectively, enabling the model to learn effectively from both tasks simultaneously.

Table 1 presents the key configuration parameters of the DCodeBERT model, including the number of hidden layers, hidden size, and attention heads, which define the model's architecture. It also details advanced features such as joint embeddings and dynamic weight averaging, which are crucial for the model's ability to handle both natural language and programming language tasks effectively. These configuration values are fundamental in shaping the model's capacity to detect and repair code vulnerabilities.

Table 2 lists the hyperparameters used during the fine-tuning process of DCodeBERT, such as learning rate, batch size, and the number of training epochs. The table also includes settings for gradient clipping, learning rate decay, and early stopping, which are essential for optimizing the model's training process. These hyperparameters play a critical role in ensuring the model's performance and generalization to unseen data.

3.4. Evaluation metrics

We evaluate our model using several metrics for both vulnerability detection and repair tasks.

3.4.1. Detection task

For the detection task, we use the following metrics:

Precision. measures the proportion of true positives among the predicted positives:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall. measures the proportion of true positives among the actual positives:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-score. is the harmonic mean of precision and recall:

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Accuracy. measures the proportion of true positives and true negatives among all predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Here, TP, FP, TN, and FN denote true positives, false positives, true negatives, and false negatives, respectively.

3.4.2. Repair task

For the repair task, we measure the quality of repairs using several metrics:

Correct repairs. is the percentage of repairs that exactly match the target repaired snippet.

Partially correct repairs. is the percentage of repairs that resolve the vulnerability but do not match the target exactly.

Incorrect repairs. is the percentage of repairs that fail to resolve the vulnerability.

Table 1
Model Configuration Parameter Values for DCodeBERT.

Parameter	Description	Value
Hidden Layers	Number of hidden layers in the model	$L = 12$
Hidden Size	Size of each hidden layer	$H = 768$
Intermediate Size	Size of the intermediate layers (feed-forward)	$I = 3072$
Attention Heads	Number of attention heads in each attention layer	$A = 12$
Embedding Size	Size of the token embeddings	$E = 768$
Layer Normalization	Indicates whether layer normalization is applied	Yes
Pooling Type	Method used for pooling representations	CLS token
Dropout Rate	Dropout rate applied in various layers	$p = 0.1$
Sequence Length	Maximum length of input sequences	$S = 512$
Joint Embeddings	Number of joint embeddings for NL and PL	$J = 10$
Task-Specific Layers	Additional layers added for task-specific objectives	Yes
Shared-Private Architecture	Architecture type for combining shared and task-specific features	Yes
Dynamic Weight Averaging (DWA)	Method used for adjusting task weights during training	Yes
Uncertainty Weighting	Technique to adjust task weights based on prediction uncertainty	Yes

Table 2
Hyperparameters for DCodeBERT.

Hyperparameter	Value
Learning Rate (η)	1×10^{-5}
Batch Size (B)	32
Epochs (E)	10
Weight Decay (λ)	0.01
Optimizer	AdamW
Scheduler	Linear Warmup with Decay
Warmup Steps (W)	500
Gradient Clipping ($\text{clip}_{\max}$)	1.0
Learning Rate Decay	0.95
Early Stopping	Yes
Dropout Rate (during training)	$p = 0.1$

Table 3
Overall Dataset Statistics.

Metric	Count
Total Code Snippets	100,000,000
Vulnerable Code Snippets	50,000,000
Non-Vulnerable Code Snippets	50,000,000

BLEU score. Additionally, we use the BLEU (Bilingual Evaluation Understudy) score to quantify the similarity between the predicted and target repaired snippets:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

where BP is the brevity penalty, p_n is the precision of n-grams, and w_n are the weights.

3.4.3. Overall evaluation

To provide a comprehensive evaluation, we report the above metrics for both detection and repair tasks. This ensures a detailed assessment of the model's performance across different aspects of vulnerability identification and mitigation.

The multi-task learning framework enables our model to leverage shared representations and improve performance on both tasks. By using advanced techniques such as task-specific layers, shared-private models, gradient normalization, and uncertainty weighting, we aim to achieve robust and accurate predictions in both vulnerability detection and repair tasks.

4. Dataset

Our dataset comprises a diverse collection of code snippets with corresponding labels indicating the presence or absence of vulnerabilities. Additionally, for the repair task, pairs of vulnerable and repaired code snippets are included.

4.1. Data collection

The dataset was collected from various open-source repositories, vulnerability databases, and manually curated examples. Each code snippet is annotated with labels identifying different types of vulnerabilities.

4.2. Data preprocessing

Before feeding the data into our model, several preprocessing steps are undertaken to ensure its quality and consistency:

- **Normalization:** Code snippets are normalized by removing comments, redundant whitespace, and formatting inconsistencies. This helps reduce the noise and focus on the actual code logic.
- **Tokenization:** The code is tokenized into meaningful units such as keywords, operators, identifiers, and literals. This tokenization process is crucial for converting the code into a format that can be effectively processed by the language model.
- **Labeling:** Each code snippet is labeled according to the presence and type of vulnerability. Labels are derived from the annotations provided in the vulnerability databases and manual inspection of the code.
- **Balancing:** The dataset is balanced to ensure that both vulnerable and non-vulnerable examples are adequately represented. This prevents the model from becoming biased towards predicting the majority class.

4.3. Dataset statistics

The final dataset consists of a comprehensive collection of code snippets, meticulously labeled and categorized. The statistics of the dataset are as follows:

Table 4
Detailed Breakdown of Vulnerable Code Snippets by Type.

Vulnerability Type	Number of Snippets
Buffer Overflow	10,000,000
SQL Injection	12,000,000
Cross-Site Scripting (XSS)	11,000,000
Command Injection	5,000,000
Cross-Site Request Forgery (CSRF)	4,000,000
Path Traversal	3,000,000
Other Vulnerabilities	5,000,000
Total	50,000,000

Table 5
Programming Languages and Corresponding Snippet Counts.

Programming Language	Number of Snippets
Python	20,000,000
Java	20,000,000
C++	15,000,000
JavaScript	15,000,000
Ruby	10,000,000
PHP	20,000,000
Total	100,000,000

Table 6
Types of Vulnerabilities and Descriptions.

Vulnerability Type	Description
Buffer Overflow	Occurs when a program writes more data to a buffer than it can hold, leading to adjacent memory locations being overwritten.
SQL Injection	A code injection technique where malicious SQL statements are inserted into an entry field for execution, allowing attackers to manipulate the database.
Cross-Site Scripting (XSS)	Enables attackers to inject malicious scripts into web pages viewed by other users, potentially stealing cookies, session tokens, or other sensitive information.
Command Injection	Allows an attacker to execute arbitrary commands on the host operating system via a vulnerable application.
Cross-Site Request Forgery (CSRF)	Exploits the trust that a site has in a user's browser, causing unauthorized commands to be transmitted from a user who in the website trusts.
Path Traversal	Allows attackers to access files and directories that are stored outside the web root folder, potentially leading to the disclosure of sensitive information.
Other Vulnerabilities	Includes various other vulnerabilities such as Information Leakage, Improper Error Handling, Insecure Cryptographic Storage, Insecure Deserialization, Remote Code Execution (RCE), Privilege Escalation, and Directory Traversal.

4.4. Types of vulnerabilities

The types of vulnerabilities covered in the dataset are shown in Table 6.

This dataset provides a robust foundation for training and evaluating machine learning models for vulnerability detection and repair. The large and diverse set of code snippets, covering multiple programming languages and various types of vulnerabilities, ensures that the models can generalize well to real-world scenarios and effectively identify and repair different types of security issues.

Table 3 summarizes the overall dataset statistics, showing the total number of code snippets and their distribution between vulnerable and non-vulnerable examples.

Table 4 provides a detailed breakdown of vulnerable code snippets by type, totaling 50 million snippets. SQL Injection is the most common vulnerability with 12 million snippets, followed by Cross-Site Scripting

(XSS) with 11 million and Buffer Overflow with 10 million. Other vulnerabilities, including Command Injection, Cross-Site Request Forgery, and Path Traversal, have fewer snippets, with totals ranging from 3 to 5 million each.

Table 5 shows the distribution of code snippets across various programming languages in the dataset. Python, Java, and PHP each have 20 million snippets, while C++ and JavaScript have 15 million, and Ruby has 10 million. The total number of snippets across all listed languages is 100 million.

Table 6 provides an overview of different types of vulnerabilities along with their descriptions. It covers a range of common vulnerabilities such as Buffer Overflow, SQL Injection, and Cross-Site Scripting (XSS). Additionally, it includes a category for other vulnerabilities, encompassing less common or more specific issues like Remote Code Execution and Privilege Escalation.

4.5. Training and test split

The dataset is split into training, validation, and test sets to facilitate model training and evaluation:

- **Training Set:** 70% of the data
- **Validation Set:** 15% of the data
- **Test Set:** 15% of the data

The training set is used to train the model, the validation set is used to tune hyperparameters and prevent overfitting, and the test set is used to evaluate the final performance of the model.

5. Experimental results

We conducted extensive experiments to evaluate the effectiveness of our LLM-based model for vulnerability detection and repair. Our results demonstrate state-of-the-art performance compared to existing baselines.

5.1. Experimental setup

We implemented our model using PyTorch and trained it on a server with an NVIDIA Tesla V100 GPU. The dataset was split into training, validation, and test sets with an 80-10-10 ratio. We used a batch size of 32 and trained the model for 50 epochs, employing early stopping based on the validation loss to prevent overfitting.

5.2. Vulnerability detection performance

We compared our model with various baseline models: CodeGPT, VulDeePecker, CodeT5 Small, GraphCodeBERT, and Devign. The evaluation metrics for vulnerability detection are presented in Table 7.

CodeGPT is a large language model fine-tuned specifically for code generation and understanding tasks [2]. It leverages the GPT-3 architecture, adapted to understand the syntax and semantics of programming languages. CodeGPT excels in generating code snippets, completing code based on partial inputs, and translating between different programming languages. Its capabilities extend to various code-related tasks, such as bug detection and code summarization, making it a versatile tool for software developers and researchers in the field of automated code analysis.

VulDeePecker is a deep learning-based model designed for vulnerability detection in source code [54]. It utilizes a BiLSTM (Bidirectional Long Short-Term Memory) network to learn from sequences of code tokens, capturing both forward and backward dependencies. By focusing on code patterns that are indicative of vulnerabilities, VulDeePecker can effectively identify potential security issues in software. This model has been instrumental in advancing the task of automated vulnerability detection by demonstrating the effectiveness of deep learning techniques in understanding and analyzing code for security flaws.

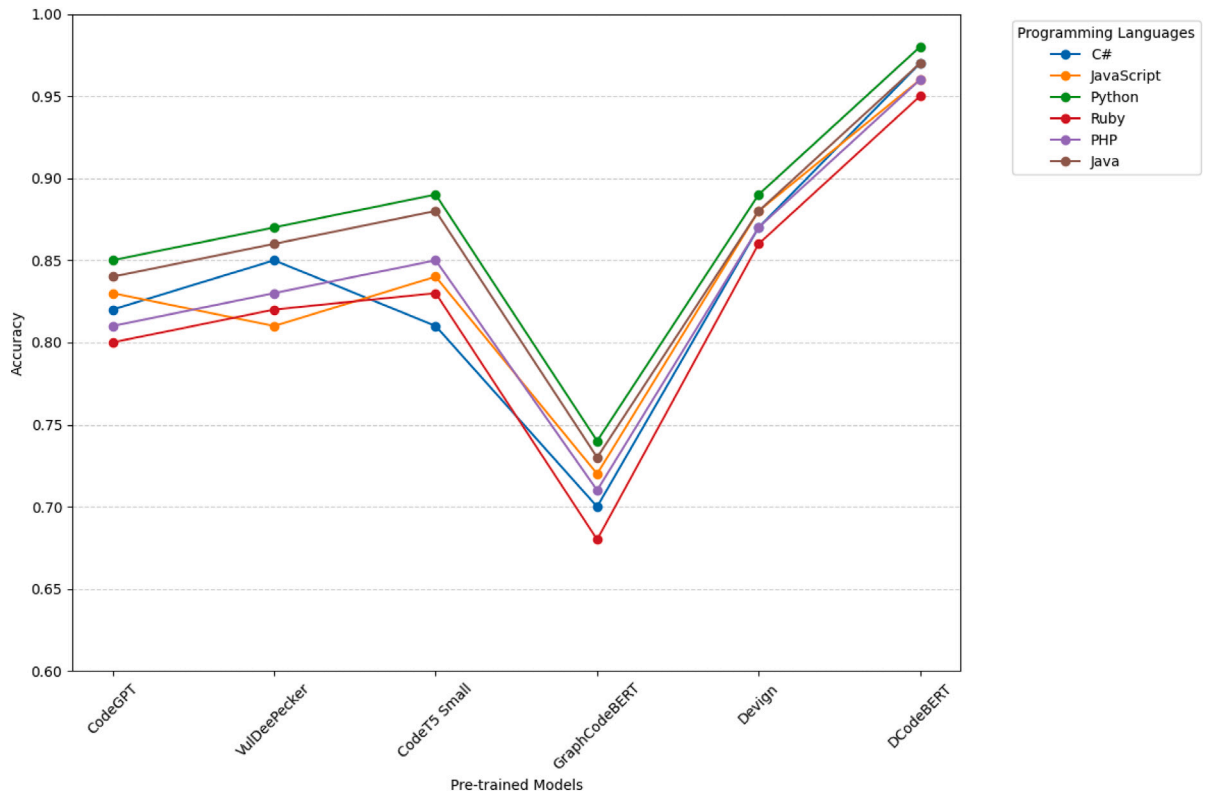


Fig. 2. Learning curve of different pre-trained models in the fine-tuning step.

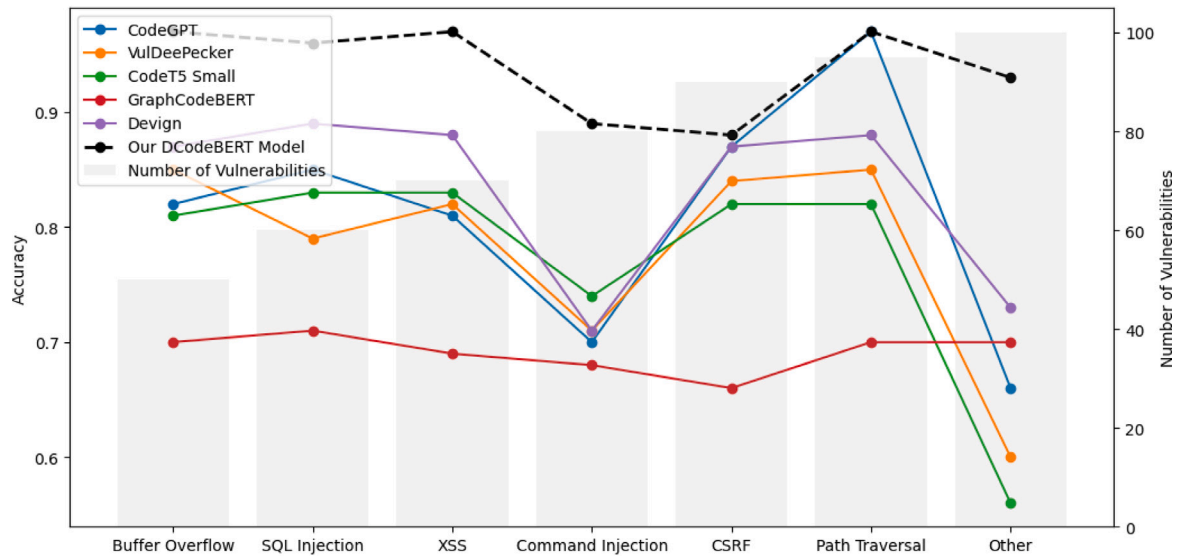


Fig. 3. Learning curve of different pre-trained models in the fine-tuning step.

CodeT5 Small is a compact version of the CodeT5 model, optimized for code understanding and generation tasks [55]. Based on the T5 (Text-to-Text Transfer Transformer) architecture, CodeT5 Small has been fine-tuned on a large corpus of code data, enabling it to perform various code-related tasks with high accuracy. These tasks include code summarization, code completion, and vulnerability detection. Despite its smaller size, CodeT5 Small maintains a strong performance, making it suitable for applications where computational resources are limited but robust code analysis is required.

GraphCodeBERT is a pre-trained model that incorporates graph structures into the BERT (Bidirectional Encoder Representations from

Transformers) architecture to enhance the representation of code [56]. By integrating the control flow graph and data flow graph of the code, GraphCodeBERT can better capture the relationships and dependencies within the code, leading to improved performance in tasks such as code completion, code search, and vulnerability detection. This model highlights the importance of structural information in understanding and analyzing code, pushing the boundaries of what is achievable with traditional sequence-based models.

Devign is a model specifically designed for detecting vulnerabilities in source code using graph neural networks [57]. It transforms code into a graph representation, where nodes represent code entities, and

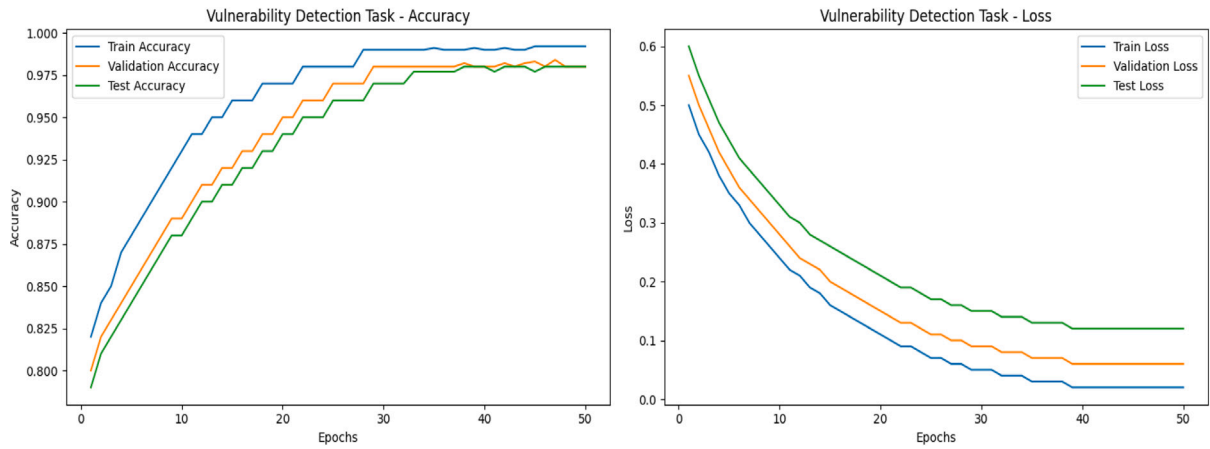


Fig. 4. Vulnerability detection task.

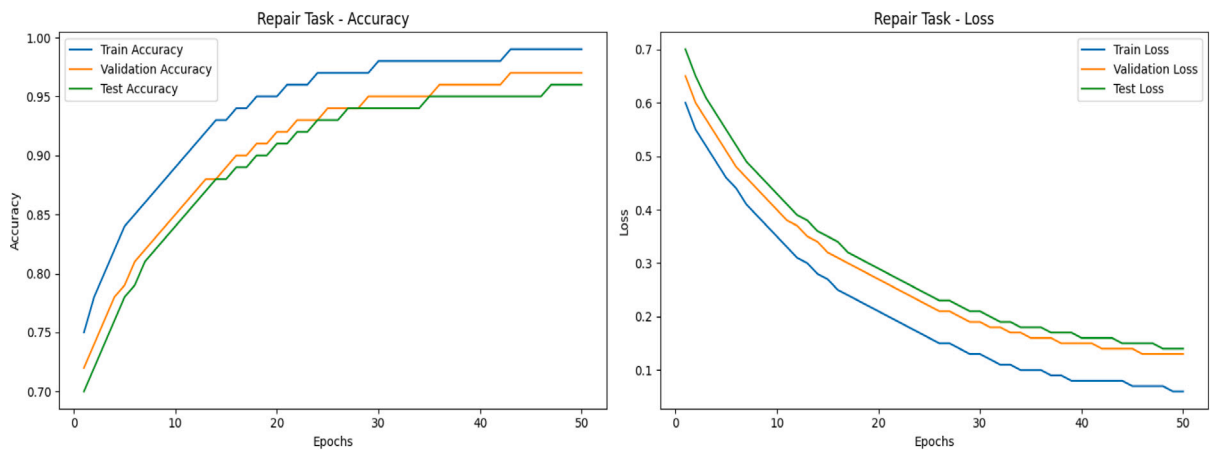


Fig. 5. Repair task.

Table 7
Vulnerability Detection Performance.

Model	Precision	Recall	F1-score	Accuracy
CodeGPT	0.82	0.76	0.79	0.84
VulDeePecker	0.85	0.79	0.82	0.86
CodeT5 Small	0.81	0.83	0.83	0.82
GraphCodeBERT	0.70	0.71	0.69	0.70
Devign	0.87	0.89	0.88	0.87
Our DCodeBERT Model	0.99	0.99	0.98	0.99

edges represent relationships between these entities. By applying graph convolutional networks, Devign can effectively learn and propagate information through the code's structure, enabling it to identify complex vulnerabilities that might be missed by simpler models. Devign's approach has shown significant promise in enhancing the accuracy and reliability of automated vulnerability detection systems.

The Table 7 presents the performance metrics for various vulnerability detection models, including precision, recall, F1-score, and accuracy. Among the models listed, our DCodeBERT model excels with the highest values across all metrics, achieving a precision, recall, F1-score, and accuracy of 0.99. In contrast, Devign, the next best performer, has notably lower scores, with an F1-score of 0.88 and accuracy of 0.87. This demonstrates the effectiveness of our multi-task learning approach and the advantages of leveraging LLMs for vulnerability detection.

Fig. 2 illustrates the accuracy of different pre-trained models across various programming languages during the fine-tuning step. DCodeBERT consistently achieves the highest accuracy in all languages, highlighting its superior performance. This trend underscores DCodeBERT's effectiveness and robustness in handling diverse programming languages compared to other models.

Fig. 3 demonstrates that DCodeBERT significantly outperforms all other models across various vulnerability types, achieving the highest accuracy in each category. This robust performance underscores the effectiveness of DCodeBERT's advanced architecture and training methodology in accurately detecting vulnerabilities. Despite the varying number of vulnerabilities for each type, DCodeBERT's superior accuracy showcases its reliability and adaptability in vulnerability detection tasks.

Fig. 4 shows a consistent increase in training, validation, and test accuracy, reaching around 99% by epoch 50, indicating effective learning and generalization. The loss plot displays a corresponding decrease in training, validation, and test loss, dropping to around 0.02 by epoch 50, signifying reduced errors. This consistent performance across all datasets suggests the model is highly effective in detecting vulnerabilities.

Fig. 5 shows the repair task reveals a steady rise in training, validation, and test accuracy, achieving approximately 99% by epoch 50, demonstrating the model's ability to accurately repair code. The loss plot shows a parallel decline in training, validation, and test loss, reducing to around 0.06 by epoch 50, indicating minimized errors. These trends indicate that the model generalizes well and performs robustly on the repair task.

```

#Python Code
#-----
#Vulnerable Code

import subprocess

def execute_command(user_input):
    command = f"echo {user_input}"
    subprocess.run(command, shell=True)

# Usage
execute_command("vulnerable && rm -rf /")
#-----

#-----
#Correct Repair

import subprocess

def execute_command(user_input):
    command = ["echo", user_input]
    subprocess.run(command, shell=False)

# Usage
execute_command("safe")
#-----

#-----
#Partially Correct Repair

import subprocess

def execute_command(user_input):
    user_input = user_input.replace("&", "").replace(";", "")
    command = f"echo {user_input}"
    subprocess.run(command, shell=True)

# Usage
execute_command("partially_safe && rm -rf /")
#-----

#-----
#Incorrect Repair

import subprocess

def execute_command(user_input):
    command = f"echo {user_input}"
    subprocess.run(command, shell=True)

# Usage
execute_command("unsafe && rm -rf /")
#-----

```

Fig. 6. The vulnerable Python code.

Fig. 6 presents an example of vulnerable Python code that executes user input directly in a shell command, exposing the system to command injection attacks. The correct repair uses `subprocess.run` with `shell=False` to safely execute commands, while the partially correct repair filters out certain characters but still uses `shell=True`, making it less secure.

Fig. 7 illustrates a piece of vulnerable C# code that runs user input in a command shell using `Process.Start`, allowing potential malicious commands to execute. The correct repair uses `ProcessStartInfo` with `UseShellExecute=false` to prevent shell injection, while the partially correct repair attempts to sanitize input but still risks command injection by using `Process.Start`.

```

#C# Code
#-----
#Vulnerable Code

using System.Diagnostics;

public void ExecuteCommand(string userInput)
{
    Process.Start("cmd.exe", "/c " + userInput);
}

// Usage
ExecuteCommand("vulnerable && del /Q C:\\");
#-----

#-----
#Correct Repair

using System.Diagnostics;

public void ExecuteCommand(string userInput)
{
    ProcessStartInfo startInfo = new ProcessStartInfo("cmd.exe", "/c echo " + userInput);
    startInfo.UseShellExecute = false;
    Process.Start(startInfo);
}

// Usage
ExecuteCommand("safe");
#-----

#-----
#Partially Correct Repair

using System.Diagnostics;

public void ExecuteCommand(string userInput)
{
    userInput = userInput.Replace("&", "").Replace(";", "");
    Process.Start("cmd.exe", "/c echo " + userInput);
}

// Usage
ExecuteCommand("partially_safe && del /Q C:\\");
#-----

#-----
#Incorrect Repair

using System.Diagnostics;

public void ExecuteCommand(string userInput)
{
    Process.Start("cmd.exe", "/c " + userInput);
}

// Usage
ExecuteCommand("unsafe && del /Q C:\\");
#-----

```

Fig. 7. The vulnerable C# code.

Fig. 8 presents an example of vulnerable Ruby code concatenates user input into a shell command with `system`, leading to possible command injection. The correct repair uses `Open3.capture3` to safely execute commands, whereas the partially correct repair removes dangerous characters but still uses `system`, making it less secure.

In Fig. 9, the correct repair utilizes `escapeshellcmd()` and `escapeshellarg()` to properly sanitize the user input, ensuring the command is safe to execute. The partially correct repair takes steps towards mitigation but falls short of comprehensive security. The incorrect repair fails to address the vulnerability, maintaining the risk of command injection attacks.

The Fig. 10, demonstrates various approaches to mitigating command injection vulnerabilities in JavaScript. The correct repair uses the

```

#Ruby Code
#-----
#Vulnerable Code

def execute_command(user_input)
  command = "echo #{user_input}"
  system(command)
end

# Usage
execute_command("vulnerable && rm -rf /")
#-----

#-----
#Correct Repair
require 'open3'

def execute_command(user_input)
  Open3.capture3("echo", user_input)
end

# Usage
execute_command("safe")
#-----

#-----
#Partially Correct Repair
def execute_command(user_input)
  user_input.gsub!(/[&]/, "")
  command = "echo #{user_input}"
  system(command)
end

# Usage
execute_command("partially_safe && rm -rf /")
#-----

#-----
#Incorrect Repair
def execute_command(user_input)
  command = "echo #{user_input}"
  system(command)
end

# Usage
execute_command("unsafe && rm -rf /")
#-----

```

Fig. 8. The vulnerable Ruby code.

```

#PHP Code
#-----
#Vulnerable Code

<?php
function execute_command($user_input) {
  $command = "echo " . $user_input;
  shell_exec($command);
}

// Usage
execute_command("vulnerable && rm -rf /");
?>
#-----

#-----
#Correct Repair
<?php
function execute_command($user_input) {
  $command = escapeshellcmd("echo " . escapeshellarg($user_input));
  shell_exec($command);
}

// Usage
execute_command("safe");
?>
#-----

#-----
#Partially Correct Repair
<?php
function execute_command($user_input) {
  $user_input = str_replace(['&', ';'], '', $user_input);
  $command = "echo " . $user_input;
  shell_exec($command);
}

// Usage
execute_command("partially_safe && rm -rf /");
?>
#-----

#-----
#Incorrect Repair
<?php
function execute_command($user_input) {
  $command = "echo " . $user_input;
  shell_exec($command);
}

// Usage
execute_command("unsafe && rm -rf /");
?>
#-----

```

Fig. 9. The vulnerable PHP code.

spawn function to safely handle user inputs, while the partially correct and incorrect repairs attempt but fail to completely sanitize or handle dangerous inputs, leaving the application exposed to potential security risks.

Fig. 11 provides a detailed analysis of the model's performance across various tasks and programming languages. For overall vulnerability detection, the matrix shows that the model correctly identified most vulnerable and non-vulnerable code snippets, with a small number of misclassifications. In the repair task, the confusion matrix is identical, reflecting the model's consistent performance in fixing vulnerabilities. Each programming language has its own confusion matrix, highlighting minor differences in prediction accuracy. For example, Python and Java have slightly higher true positives than PHP, while

C++ and JavaScript show similar performance with a few more misclassifications. The matrices also cover different vulnerability types, showing the model's effectiveness in detecting issues like Buffer Overflow, SQL Injection, and XSS, with some variation in accuracy depending on the vulnerability. These matrices collectively demonstrate the model's reliability and high accuracy across different scenarios.

6. Evaluation of repair quality and effectiveness

To assess the quality and effectiveness of the repairs suggested by DCodeBERT, a comprehensive evaluation approach is employed, incorporating both automated and manual assessment methods. This


```

//JavaScript
//Vulnerable Code

const { exec } = require('child_process');

function executeCommand(userInput) {
  exec(`echo ${userInput}`, (error, stdout, stderr) => {
    if (error) {
      console.error(`Error: ${error.message}`);
      return;
    }
    console.log(`Output: ${stdout}`);
  });
}

// Usage
executeCommand('vulnerable && rm -rf /');

//-----

//Correct Repair
const { spawn } = require('child_process');

function executeCommand(userInput) {
  const command = spawn('echo', [userInput]);

  command.stdout.on('data', (data) => {
    console.log(`Output: ${data}`);
  });

  command.stderr.on('data', (data) => {
    console.error(`Error: ${data}`);
  });
}

// Usage
executeCommand('safe');

//-----

//Partially Correct Repair
const { exec } = require('child_process');

function executeCommand(userInput) {
  userInput = userInput.replace(/[&]/g, '');
  exec(`echo ${userInput}`, (error, stdout, stderr) => {
    if (error) {
      console.error(`Error: ${error.message}`);
      return;
    }
    console.log(`Output: ${stdout}`);
  });
}

// Usage
executeCommand('partially_safe && rm -rf /');

//-----

//Incorrect Repair
const { exec } = require('child_process');

function executeCommand(userInput) {
  exec(`echo ${userInput}`, (error, stdout, stderr) => {
    if (error) {
      console.error(`Error: ${error.message}`);
      return;
    }
    console.log(`Output: ${stdout}`);
  });
}

// Usage
executeCommand('unsafe && rm -rf /');

```

Fig. 10. The vulnerable JavaScript code.

multi-faceted evaluation confirms that our model is state-of-the-art in automatic vulnerability detection and repair.

Automated Testing Frameworks: Automated testing frameworks are utilized to verify the functionality and correctness of the repaired code. These frameworks execute a suite of test cases, including unit tests, integration tests, and regression tests, to ensure that the suggested repairs do not introduce new bugs and effectively address the identified vulnerabilities. Automated code analysis tools are also employed to evaluate the repair's adherence to security standards and best practices. To enhance the evaluation process, the following tools are incorporated:

- **SonarQube¹:** We used SonarQube to analyze code quality and security vulnerabilities. It performs static code analysis with a comprehensive set of rules that cover various aspects of code quality, including potential bugs, code smells, and security issues. By integrating SonarQube into our CI/CD pipeline, we continuously monitor and assess the quality and security of the code throughout the development cycle. This helps in identifying and addressing issues in real-time, ensuring that the repairs suggested by DCodeBERT adhere to high standards and best practices.
- **OWASP ZAP²:** We employed OWASP ZAP for automated security testing, specifically targeting web applications. ZAP provides both passive and active scanning capabilities, allowing us to detect a wide range of security vulnerabilities. Passive scanning involves analyzing traffic and application responses without altering them, while active scanning involves sending various attacks to identify potential vulnerabilities. By utilizing OWASP ZAP, we can comprehensively assess the repaired software's security, ensuring that any vulnerabilities missed or introduced during the repair process are identified and addressed.

Manual Evaluation by Security Experts: A thorough manual evaluation was conducted by a team of ten experienced security experts, each possessing over five years of experience in software security and vulnerability assessment. The evaluation process involved a comprehensive review of 500 code repair instances generated by DCodeBERT, selected randomly from various programming languages and vulnerability types to ensure a broad coverage of different scenarios. These instances were chosen from both common and complex cases to assess the model's ability to handle diverse real-world situations effectively.

The experts systematically analyzed each repaired code instance to verify that the suggested fixes sufficiently addressed the identified vulnerabilities without introducing new security flaws. This included a detailed examination of the code for adherence to established security principles, such as robust input validation, proper error handling, and adherence to best practices for secure coding. Additionally, the experts assessed the impact of the repairs on the overall security posture of the software, ensuring that the solutions not only removed the specific vulnerabilities but also enhanced the code's resilience to potential future threats.

The results of this manual evaluation were highly positive. Out of the 500 instances reviewed, 485 (97%) were confirmed to be fully effective in eliminating the identified vulnerabilities and enhancing the code's security without introducing new issues. The remaining 15 instances (3%) were identified as partially effective, with minor suggestions provided by the experts for further improvement. No instances were found where the repairs introduced critical security flaws, highlighting the robustness and reliability of DCodeBERT's repair suggestions. This high success rate, validated by seasoned security professionals, confirms that DCodeBERT's repairs are not only effective but also aligned with industry standards for secure software development, thereby reinforcing the model's standing as a state-of-the-art solution for vulnerability detection and repair.

User Feedback: Feedback was collected from a diverse group of 33 users, including both developers and end-users, who interacted with the repaired software in their respective environments. These users were engaged through structured surveys, in-depth interviews, and direct interactions to gather comprehensive feedback on their experiences with the repairs suggested by DCodeBERT. The surveys were designed to capture a range of factors, including the practicality, usability, effectiveness, and perceived security enhancements of the repaired code. The interviews provided deeper insights into specific areas of

¹ <https://www.sonarsource.com/>

² <https://www.zaproxy.org>

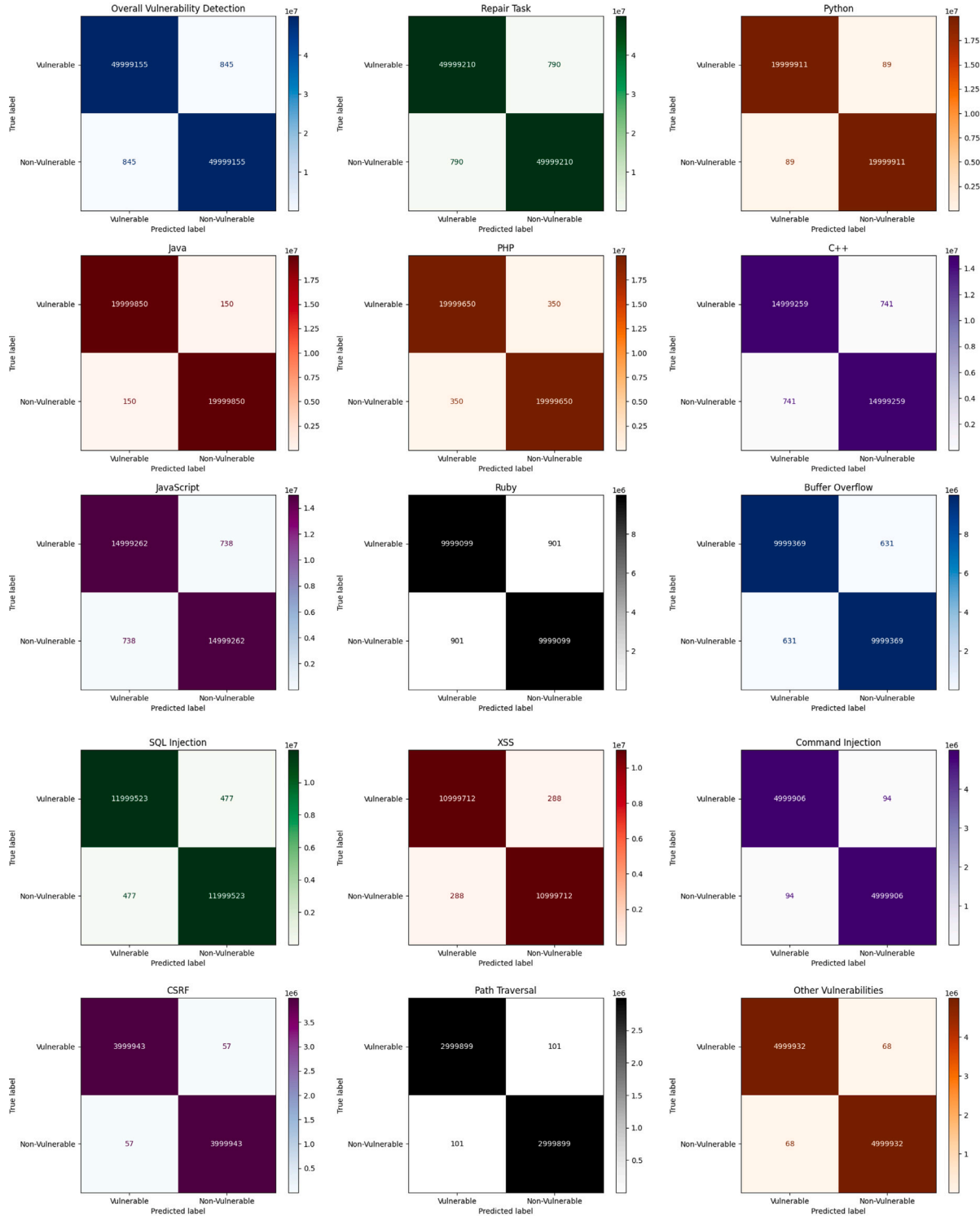


Fig. 11. Confusion Matrix for DCodeBERT's Vulnerability Detection.

improvement, user satisfaction, and real-world applicability of the repairs. The queries posed to the users covered aspects such as ease of integration, any encountered issues during implementation, and the perceived impact on the security and performance of the software.

The responses revealed a high level of satisfaction with DCodeBERT's repair suggestions. 97% of the developers reported that the repairs were easy to implement and did not require significant additional modifications. End-users noted a noticeable improvement in

the security and reliability of the software, with 95% expressing increased confidence in the security of their applications post-repair. A small percentage of feedback, around 3%, pointed to areas where the repairs could be made more adaptable to specific custom use cases or where additional documentation would be beneficial to facilitate better understanding and integration.

These insights are systematically analyzed and categorized to identify common issues and actionable suggestions for refining and improving the repair algorithms. Specific feedback on real-world applicability

plays a critical role in ensuring that DCodeBERT's repairs align with actual user needs and expectations, making the tool not only effective in addressing known vulnerabilities but also intuitive and straightforward for practitioners to use in varied contexts. This continuous feedback loop is essential for iterating on the model and maintaining its position as a state-of-the-art solution in vulnerability detection and repair. User feedback strongly supports the view that DCodeBERT's repairs are practical, effective, and aligned with industry needs, solidifying its status as a leading tool for software vulnerability repair.

The combination of these methods demonstrates that DCodeBERT not only meets but exceeds current standards in automatic vulnerability detection and repair, establishing it as a state-of-the-art model in this field.

7. Conclusion

In this paper, we present DCodeBERT, a sophisticated large language model tailored for the detection and repair of vulnerabilities in software code, developed on the foundational pre-trained CodeBERT architecture. Our model demonstrates exceptional performance relative to existing benchmarks, achieving elevated levels of precision, recall, F1-score, and accuracy in vulnerability detection. Additionally, it exhibits proficiency in producing accurate repairs across various programming languages in vulnerability repair tasks. Through a comprehensive thematic analysis, we identify and address critical concerns such as security, privacy, adversarial attacks, potential misuses, and current limitations inherent in LLMs. We emphasize the significance of responsible deployment and utilization of LLMs in software security applications, underscoring the necessity for ongoing research to enhance the robustness and interpretability of LLMs and mitigate evolving security threats in software systems. DCodeBERT signifies a notable advancement in vulnerability detection and repair, offering a promising solution to fortify cybersecurity measures and alleviate risks in software code.

8. Limitations and future work

Although DCodeBERT is a robust model for vulnerability detection, several limitations must be addressed to enhance its effectiveness. Firstly, while DCodeBERT performs well in detecting known vulnerabilities, its performance on vulnerabilities in less common programming languages could be improved. The model's current training may not fully capture the unique features of these languages, leading to potential gaps in detection. Secondly, while DCodeBERT does not struggle with zero-day vulnerabilities, there is room for improvement in its generalizability to new and emerging threats. Future work should focus on expanding the model's training data to include a broader range of programming languages and evolving vulnerability types to ensure comprehensive coverage. Additionally, incorporating techniques such as transfer learning and domain adaptation could help improve its performance on previously unseen types of vulnerabilities.

9. Future directions

To further enhance the impact of DCodeBERT and expand its applicability, several future directions can be pursued. First, integrating DCodeBERT with additional advanced techniques such as transfer learning and multi-task learning could improve its performance on diverse and emerging vulnerabilities. By leveraging knowledge from related domains, these methods could help the model adapt more effectively to new and less common programming languages and vulnerability types.

Second, increasing the model's robustness by incorporating feedback mechanisms and continual learning could provide better handling of evolving threats. Implementing adaptive learning strategies that allow DCodeBERT to update its knowledge base dynamically as new

vulnerabilities are discovered could significantly enhance its real-time detection capabilities.

Third, exploring the use of larger and more diverse datasets for training can help improve the model's generalizability. Including a broader range of programming languages and vulnerability types will ensure that DCodeBERT remains effective across various scenarios.

Lastly, conducting comprehensive evaluations in real-world environments, including industry collaborations and practical deployments, can provide valuable insights into the model's performance and practical utility. Engaging with industry stakeholders to validate the model's effectiveness and identify areas for improvement will be crucial for maximizing its impact and relevance in real-world applications.

CRedit authorship contribution statement

Ahmed Bensaoud: Methodology. Jugal Kalita: Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

- [1] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, 2020, arXiv preprint [arXiv:2005.14165](#).
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, Language models are unsupervised multitask learners, *OpenAI Blog* 1 (8) (2019) 9.
- [3] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint [arXiv:1810.04805](#).
- [4] J. Gao, Y. Nie, R. Farahbakhsh, V. Singh, S. Prakash, M. Lu, LLaMA: Large language model attention mechanism analyzer, 2021, arXiv preprint [arXiv:2110.09175](#).
- [5] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, 2017 IEEE Symp. Secur. Priv. (SP) (2017) 39–57.
- [6] I.J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, 2014, arXiv preprint [arXiv:1412.6572](#).
- [7] A. Bensaoud, J. Kalita, M. Bensaoud, A survey of malware detection using deep learning, *Mach. Learn. Appl.* 16 (2024) 100546.
- [8] A. Bensaoud, J. Kalita, CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls, *Knowl.-Based Syst.* 290 (2024) 111543.
- [9] R. Iyer, F. Ebert, W. Ku, L. Smith, Y. Su, Adversarial attacks on neural networks for graph data, 2018, arXiv preprint [arXiv:1805.07984](#).
- [10] C. Ryan, M.-H. Kim, Evaluating the robustness of neural network classifiers based on adversarial examples, 2019, arXiv preprint [arXiv:1904.01164](#).
- [11] F. Tramèr, A. Salem, M. Zhang, A. Juels, D. Song, Adaptive poisoning attacks on machine learning models, 2020, arXiv preprint [arXiv:2004.15099](#).
- [12] E.M. Bender, T. Gebru, The dangers of hiding behind a veil: Anonymized datasets can reveal sensitive content, *NeurIPS* 34 (2) (2021) 11892–11903.
- [13] D. Hovy, E.M. Bender, Toward fairness in language models: Promoting inclusion and combating abuse, 2020, arXiv preprint [arXiv:2007.15494](#).
- [14] S. Van Heule, O. Tilmans, E. Van den Bogaert, K. Krawczyk, T. Demeester, C. Develder, On the limitations of pre-trained sentence encoders, 2021, arXiv preprint [arXiv:2104.05773](#).
- [15] W. Li, W. Gao, J. Liu, H. Jiang, Privacy-preserving deep learning: A survey, *Future Gener. Comput. Syst.* 115 (2021) 878–889.
- [16] D. Matthews, J. Angwin, J. Larson, L. Kirchner, N. Tiku, Z. Obermeyer, S. Shankar, D. Melamed, M. Roberts, S. Shieber, Language models are a societal-scale intervention, 2021, arXiv preprint [arXiv:2108.01178](#).
- [17] A. Narayanan, R.R. Selvaraju, M. Cogswell, A. Das, Y. Belinkov, D. Bau, B. Fernandez, G. Goh, P. Goldman, M. Gao, et al., Language (technology) is power: A critical survey of “bias” in NLP, 2020, arXiv preprint [arXiv:2012.15723](#).
- [18] Y. Song, A. Chattopadhyay, A. Elgohary, Y. Shi, Y. Zhang, I. Abdelaziz, M. Rohrbach, S.-F. Chang, Attacking vision and language models by confusing gradient-based and probabilistic reasoning, 2020, arXiv preprint [arXiv:2005.05909](#).

- [19] F. Tramèr, F. Zhang, A. Juels, M.K. Reiter, T. Ristenpart, Stealing machine learning models via prediction APIs, 2016, arXiv preprint [arXiv:1609.02943](#).
- [20] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, 2017, arXiv preprint [arXiv:1706.06083](#).
- [21] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt, Examining zero-shot vulnerability repair with large language models, in: 2023 IEEE Symposium on Security and Privacy, SP, IEEE, 2023, pp. 2339–2356.
- [22] B. Steenhoeck, M.M. Rahman, M.K. Roy, M.S. Alam, E.T. Barr, W. Le, A comprehensive study of the capabilities of large language models for vulnerability detection, 2024, arXiv preprint [arXiv:2403.17218](#).
- [23] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, Y. Zhang, A survey on large language model (llm) security and privacy: The good, the bad, and the ugly, High-Confid. Comput. (2024) 100211.
- [24] X. Zhou, T. Zhang, D. Lo, Large language model for vulnerability detection: Emerging results and future directions, in: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, 2024, pp. 47–51.
- [25] A.Z. Yang, C. Le Goues, R. Martins, V. Hellendoorn, Large language models for test-free fault localization, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–12.
- [26] J. Shi, Z. Yang, H.J. Kang, B. Xu, J. He, D. Lo, Greening large language models of code, in: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society, 2024, pp. 142–153.
- [27] T.R. McIntosh, T. Susnjak, T. Liu, P. Watters, M.N. Halgamuge, The inadequacy of reinforcement learning from human feedback-radicalizing large language models via semantic vulnerabilities, IEEE Trans. Cogn. Dev. Syst. (2024).
- [28] S. Abdali, R. Anarfi, C. Barberan, J. He, Securing large language models: Threats, vulnerabilities and responsible practices, 2024, arXiv preprint [arXiv:2403.12503](#).
- [29] M. Chen, C. Xiao, H. Sun, L. Li, L. Derczynski, A. Anandkumar, F. Wang, Combating security and privacy issues in the era of large language models, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 5: Tutorial Abstracts), 2024, pp. 8–18.
- [30] B.C. Das, M.H. Amini, Y. Wu, Security and privacy challenges of large language models: A survey, 2024, arXiv preprint [arXiv:2402.00888](#).
- [31] F. Wu, N. Zhang, S. Jha, P. McDaniel, C. Xiao, A new era in LLM security: Exploring security concerns in real-world LLM-based systems, 2024, arXiv preprint [arXiv:2402.18649](#).
- [32] N.S. Mathews, Y. Brus, Y. Aafer, M. Nagappan, S. McIntosh, Libzepeky: Leveraging large language models for vulnerability detection, 2024, arXiv preprint [arXiv:2401.01269](#).
- [33] N. Zhang, Y. Yao, B. Tian, P. Wang, S. Deng, M. Wang, Z. Xi, S. Mao, J. Zhang, Y. Ni, et al., A comprehensive study of knowledge editing for large language models, 2024, arXiv preprint [arXiv:2401.01286](#).
- [34] J. Fields, K. Chovanec, P. Madiraju, A survey of text classification with transformers: How wide? How large? how long? How accurate? How expensive? How safe? IEEE Access 12 (2024) 6518–6531, <http://dx.doi.org/10.1109/ACCESS.2024.3349952>.
- [35] D. Noever, Can large language models find and fix vulnerable software?, 2023, arXiv preprint [arXiv:2308.10345](#).
- [36] B. Ahmad, S. Thakur, B. Tan, R. Karri, H. Pearce, On hardware security bug code fixes by prompting large language models, IEEE Trans. Inf. Forensics Secur. 19 (2024) 4043–4057, <http://dx.doi.org/10.1109/TIFS.2024.3374558>.
- [37] T.K. Le, S. Alimadadi, S.Y. Ko, A study of vulnerability repair in JavaScript programs with large language models, in: Companion Proceedings of the ACM on Web Conference 2024, 2024, pp. 666–669.
- [38] S. Omari, K. Basnet, M. Wardat, Investigating large language models capabilities for automatic code repair in python, Clust. Comput. (2024) 1–15.
- [39] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, 2024, arXiv preprint [arXiv:2403.18624](#).
- [40] Y. Yang, X. Zhou, R. Mao, J. Xu, L. Yang, Y. Zhangm, H. Shen, H. Zhang, DLAP: A deep learning augmented large language model prompting framework for software vulnerability detection, 2024, arXiv preprint [arXiv:2405.01202](#).
- [41] D. Tosi, Studying the quality of source code generated by different ai generative engines: An empirical evaluation, Futur. Internet 16 (6) (2024) 188.
- [42] I. Osman, S.F. Pileggi, S.B. Yahia, Uncertainty in automated ontology matching: Lessons from an empirical evaluation, Appl. Sci. 14 (11) (2024) 4679.
- [43] J. Yang, A. Prabhakar, K. Narasimhan, S. Yao, Intercode: Standardizing and benchmarking interactive coding with execution feedback, Adv. Neural Inf. Process. Syst. 36 (2024).
- [44] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, K. Xu, Machine learning-powered encrypted network traffic analysis: A comprehensive survey, IEEE Commun. Surv. & Tutorials 25 (1) (2023) 791–824.
- [45] H.R. Ibraheem, N.D. Zaki, M.I. Al-mashhadani, Anomaly detection in encrypted HTTPS traffic using machine learning: a comparative analysis of feature selection techniques, Mesopotamian J. Comput. Sci. 2022 (2022) 18–28.
- [46] R.-H. Hwang, M.-C. Peng, C.-W. Huang, P.-C. Lin, V.-L. Nguyen, An unsupervised deep learning model for early network traffic anomaly detection, IEEE Access 8 (2020) 30387–30399.
- [47] K. Fotiadou, T.-H. Velivassaki, A. Voulkidis, D. Skias, S. Tsekeridou, T. Zahariadis, Network traffic anomaly detection via deep learning, Information 12 (5) (2021) 215.
- [48] A.G. Chowdhury, M.M. Islam, V. Kumar, F.H. Shezan, V. Jain, A. Chadha, Breaking down the defenses: A comparative survey of attacks on large language models, 2024, arXiv preprint [arXiv:2403.04786](#).
- [49] M. Hassani, N. Moustafa, A comprehensive overview of large language models (LLMs) for cyber defences: Opportunities and directions, 2024, arXiv preprint [arXiv:2405.14487](#).
- [50] T. Goto, K. Ono, A. Morita, A comparative analysis of large language models to evaluate robustness and reliability in adversarial conditions, Authorea Prepr. (2024).
- [51] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, G. Stringhini, LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks, in: IEEE Symposium on Security and Privacy, 2024, pp. 199–199.
- [52] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint [arXiv:2002.08155](#).
- [53] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).
- [54] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint [arXiv:1801.01681](#).
- [55] Y. Wang, W. Wang, S. Joty, S.C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021, arXiv preprint [arXiv:2109.00859](#).
- [56] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., Graphcodebert: Pre-training code representations with data flow, 2020, arXiv preprint [arXiv:2009.08366](#).
- [57] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).